

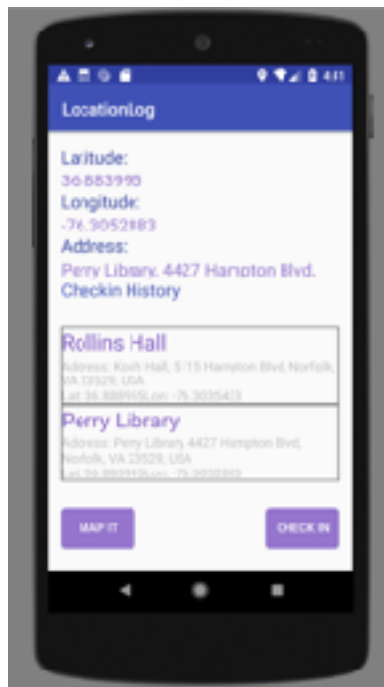
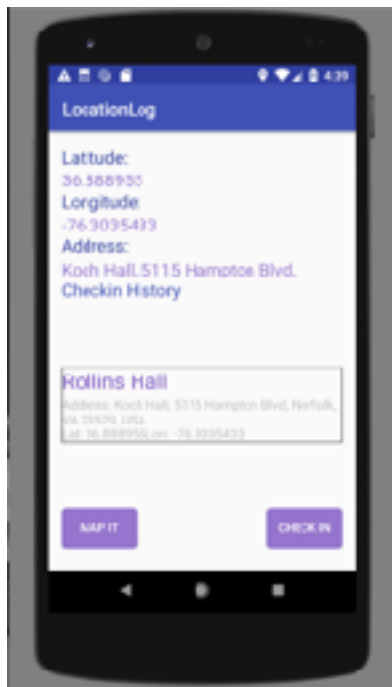
Report: Location Logger

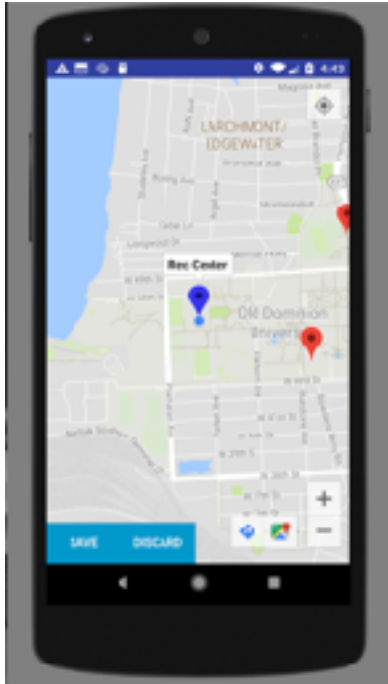
Justin Whitlock

1. Most of part one of the assignment takes place in the initial launch screen.
 - A. Upon launch, the last known location is obtained using the `getFusedLocationProviderClient` and a function called `startLocationStuff`. The Lat Lon are displayed at the top of the Main Activity.
 - B. Once the location has been found, an `IntentService` is launched which fetches the address in the background.
 - C. `mLocationCallback` is defined in `onCreate()`. It passes the location to a function called `setData()` which calls the `intentService` for the address and handles updating the current location. It also updates the current `Checkin` Object.
 - D. `Checkin` appears bottom right of the `MainActivity`. It launches an alert `Dialog` to handle each checkin and name. Checkins are stored in `checkinDB` using the current `CHECKIN` object. The Database consists of two tables: `checkins` and `places`. The list of checkins appears below `Checkin History` in the `MainActivity`.
 - E. When the alert dialog pops up, it gives the user the opportunity to assign a name to the checkin. This is saved in the `places` table while the checkin holds the `PID` of the associated place in its table.
 - F. When a location is updated, the location is compared to the `places` table to see if it is within 30 meters of a known location. This occurs in the `setData()` function, where it calls `checkForName()` and uses `Location.distanceTo()` to check if its nearby. When the user clicks checkin, the probable checkin name is offered as a name for this location. The user can accept the name, or decline it and enter a new name. This occurs in the alert dialog. The displayed checkin list uses the `arrayList` `checkinList` and uses the `PID` associated with each checkin to fetch place names for each checkin from the `places` table. Each place is stored only once, and each checkin is stored once.
2. On the bottom left of the `MainActivity` there is a `Map It` button which takes you to the `MapsActivity`.
 - A. The current `Location` is obtained using `locationManager.getLastKnownLocation(bestProvider)`. Using the returned location, the camera is moved in a function called `locationUpdate()`. Buttons and map controls are setup using `UISettings` in the `onMapReady()` function.
 - B. Checkins and Places are plotted on the map in the function `PopulateMap()` which is called in `OnMapReady()` and whenever the list of places is updated.

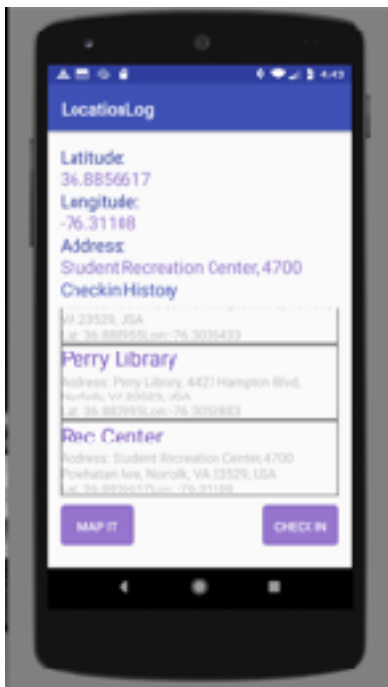
- C. To add a marker, there is a button in the bottom left. This starts a dialog which allows the user to add a name. I didn't use the on map click to do this because the onMapClick wasn't behaving in the emulator. It kept creating new points when I was trying to drag other markers or interact with the UI controls. After the Dialogue box clears, a Blue Marker appears on the map at the users current location. They are then able to drag the marker around the map to place it. Once the marker is placed, the user can click save, to add the place to the places table in the database, or discard to remove the marker. Once save is clicked, the Marker turns red, and the save and discard buttons are replaced by the add button.
3. I was unable to complete the final portion of the assignment. I do not have an android phone and using the emulator does not enable you to log network responsiveness or location accuracy as there is no network.

Checkin at Koch Hall (accidentally named it Rollins) and Perry Library.

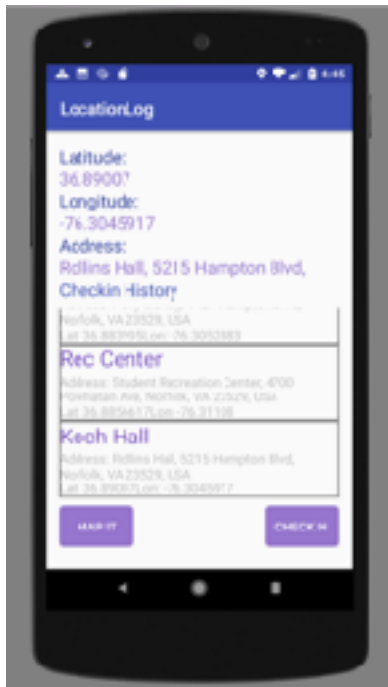




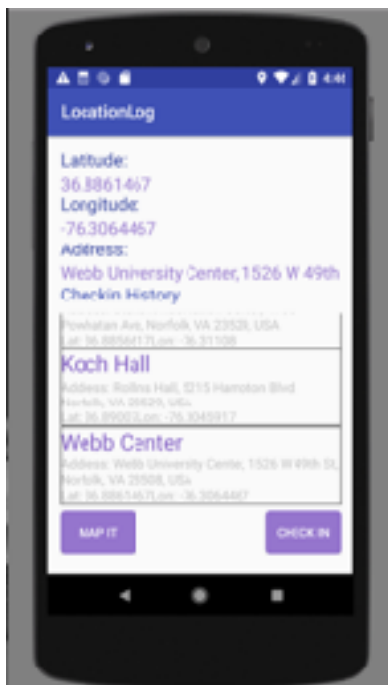
Pin added for the Rec Center



After adding the Pin, returned to the MainActivity and checked into the Rec Center, which was suggested as a place because the Rec Center was added to the places table in the MapsActivity.



Checked into Rollins Hall (named it Koch just because).



Checked into the Webb Center.